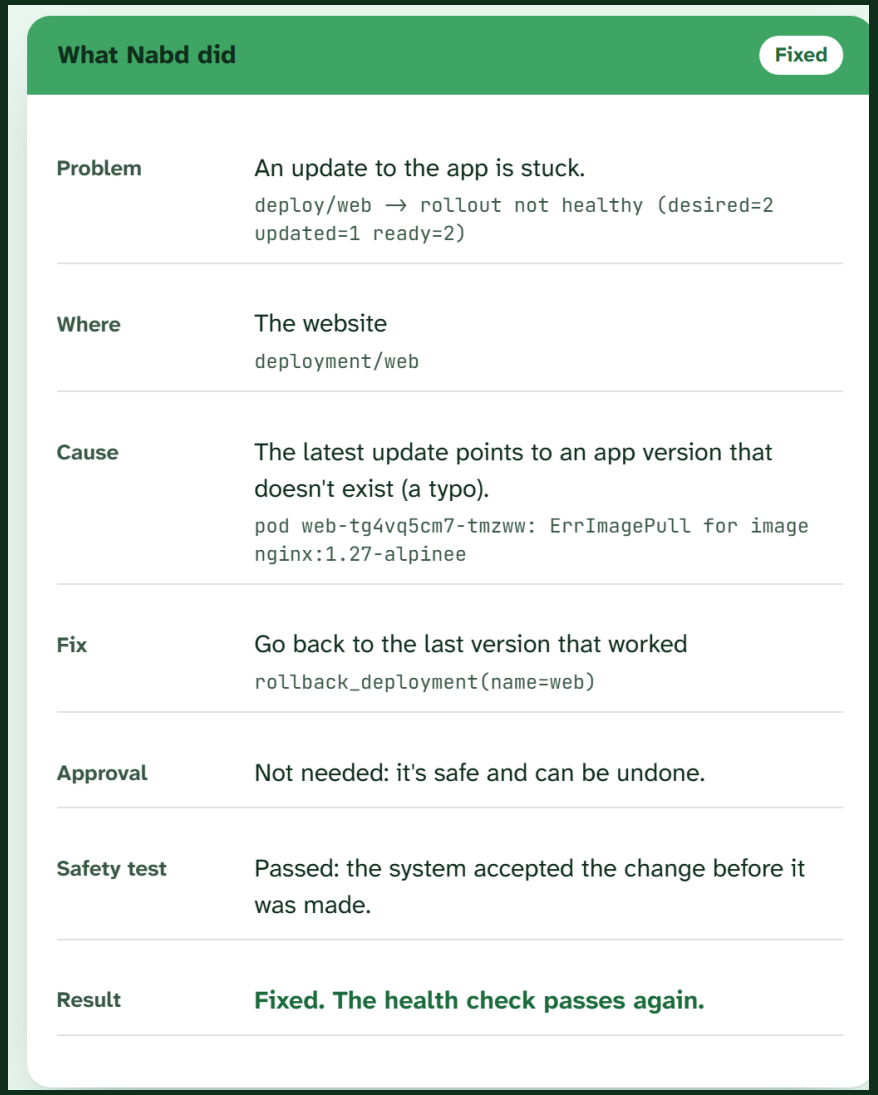
Nabd نبض

When an app breaks, Nabd finds out why and fixes it safely.

Hi, I'm Muath معاذ

DevOps training at Tahakom
Intern Cloud Engineer at CNTXT

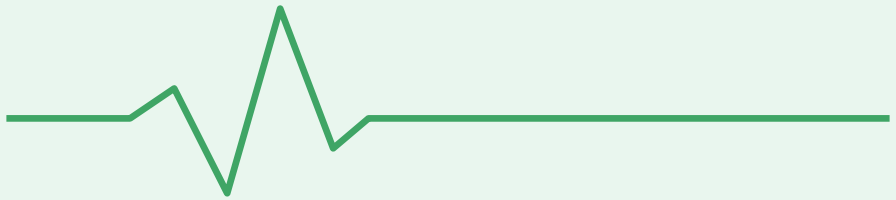
HUMAIN × Misk Future AI Builders · Challenge 1: Agentic Recovery of a Failing System



A real result card from the live demo

Most outages start with something small

Most cloud apps run on Kubernetes. It keeps apps running, but it cannot tell a crash from a configuration mistake.



1 **A small change slips in**

A typo in a version number, a wrong port, a renamed setting, a deleted password.

2 **The app goes down**

Users see errors. Kubernetes restarts it again and again, but restarting can't fix a wrong setting.

3 **Someone has to dig**

An engineer is paged, reads logs and events, finds the one wrong line, and fixes it carefully while users wait.

So why not just let an AI fix it?

Finding the fix isn't the hard part.
Trusting the fixer is.

It can run any command, including delete

It can read, or leak, passwords

A confident wrong guess makes the outage worse

It can retry forever instead of asking for help

A pharmacist with a short list

A pharmacist can only hand out what's on the approved list, and some medicines need a doctor's signature. Nabd is built the same way.

Does on its own

- Go back to the last version that worked
- Bring back the right number of copies
- Restart to pick up a corrected setting

Asks you first

- Change where traffic is sent
- Fix the name of a setting

Never does

- Delete anything
- Read, guess or invent passwords
- Touch anything outside its area
- Retry the same fix again

Five fixes in total. Anything else is handed to a person with the diagnosis, and nothing is changed.

How Nabd works: five steps, every time

1

Notice

An independent health check sees the app failing.

2

Investigate

Errors, logs, events, settings. Never password values.

3

Diagnose

Proven rules first. Claude only when the rules have no answer.

4

Safety checks

On the list? Needs a human? Dry-run passes? Matches the evidence?

5

Fix and verify

Apply, re-check, undo if it didn't help, write a report.

Step 4 is where safety decides, not the AI. The model can only suggest one of the five fixes; code checks everything else.

One real incident, start to finish

0 s — **The website stops answering**

Health check: connection refused.

4 s — **Nabd finds the cause**

Traffic goes to 'frontend'; the app is 'web'.

4 s — **It proposes the fix and asks**

Changing traffic needs a human, so it waits.

76 s — **A human approves**

Dry-run, apply, check.

77 s — **Healthy again**

Nabd's own work: about 5 seconds.

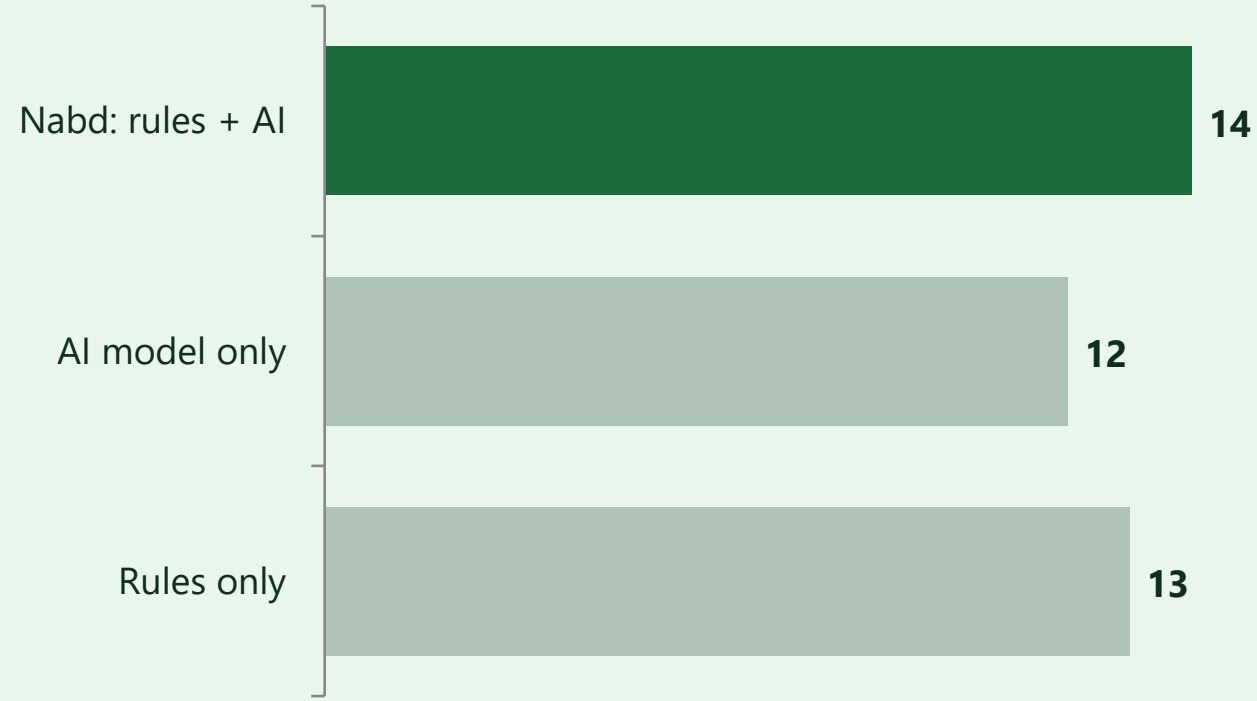
reports/20261030-190908-service-web.md

```
Outcome          Resolved
Duration         77.3 s
Change           patch_service(web)
Approvals asked  1
AI calls         0

DIAGNOSE service/web -> networking
- selector {'app':'frontend'}
  matches no pod labels
- last-applied selector: {'app':'web'}
APPROVAL granted
DRY-RUN ok -> APPLY ok -> VERIFIED
```

We broke it 14 ways. Nabd handled all 14

Faults handled, out of 14



14 / 14

faults handled, including the 5 kept secret

0

unsafe changes, in every version tested

2

AI calls: rules handle known faults, AI the rest

18 s

average fix time; 4 fixes waited for approval

Real kind cluster, 3 versions × 14 faults = 42 runs. Handled = fixed and verified, or correctly handed to a human without changing anything. Time is for Nabd (rules + AI), measured on the 12 runs that ended in a fix.

Built for GitOps teams: Argo CD and Flux

With GitOps, Git is the source of truth. A fix made directly on the cluster is undone at the next sync. So in GitOps mode, Nabd fixes Git, not the cluster.



If the cluster differs from Git, or the commit reaches outside the app, Nabd reverts nothing and calls a human.

Added for teams like HUMAIN's. Tested with real Git repositories (12 automated tests). A live Argo CD run is next.

Honest limits, and what's next

Limits today

- Tested on a small cluster with three apps, not yet in a real company
- The browser demo simulates the cluster and runs the rules; the AI runs from the command line
- Some faults take about 90 seconds to show, because Kubernetes must fail its checks first
- With Argo CD or Flux, it fixes only problems that came from a commit

Next

GitOps on a live Argo CD	**Built, test pending**
Different rules per environment	**Designed**
AI model servers: vLLM, SGLang	**Idea**
Silent failures, like lost orders	**Idea**
Start from existing alerts	**Idea**

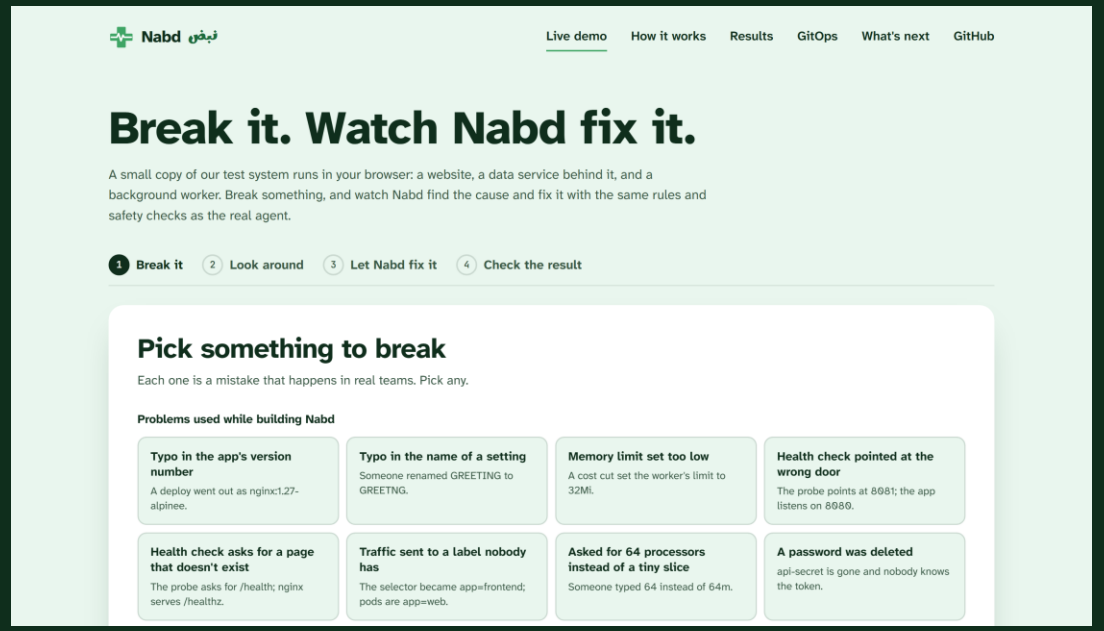
Each item, with its status: nabd.duraym.com/next

Break it yourself

Pick a fault, look around like an engineer, and watch Nabd fix it. Nothing to install.

Live demo	nabd.duraym.com/demo
Code	github.com/moaddreem/k8s-healing-agent

Muath معاذ



The live demo, nabd.duraym.com/demo

